

: before anything I need you to ingest all contents of the two top files and the corpus folders files as well

 OPERATOR_README.md

 OPERATOR_MAP.md

 __NovusÆxorpheus_LIVING_MASTER_CANON

Once you have a concrete understanding of my Enterprise stack and user intent confirm with me in a grill session ala Matt pocock's famous five grill with docs skills style of building and through discussion you we will plan out and you will make specific solution requests that I will have to verify before anything ever gets built

I am building a sovereign Orchestrator & Cross-Auditor Agent for the NovusÆxenti / NovusÆcopia ecosystem.

All foundational architecture specifications, file trees, and subsystem readmes have been committed to Google Drive:

- Master Guide: OPERATOR_README.md

(<https://docs.google.com/document/d/1MoktPebBEqcxNcDPBcgiShXdGOZr9CcdB9y0-VWJLCY/edit?usp=drivesdk>)

- Master Map: OPERATOR_MAP.md

(<https://docs.google.com/document/d/1ws8wAJ4SF0K6VkzA0gEOGpAtccTw1uK9e-B3NVDkcig/edit?usp=drivesdk>)

- Master Canon Folder: __NovusÆxorpheus_LIVING_MASTER_CANON

(<https://drive.google.com/drive/folders/1xv0yKrUnIzUqLBUfDIIHo0uWFv-KqTBV>)

PROJECT OBJECTIVE:

Create a lightweight Orchestrator / Cross-Auditor Agent that works alongside Claude Code CLI to monitor, audit, and enforce compliance on every prompt and tool call.

CRITICAL CONSTRAINTS & WORKFLOW:

- Do NOT generate or assume the entire codebase upfront. We will review and build this collaboratively, step by step, in-session.

- I already have custom code in my fork specifically designed for this setup, which intercepts tool calls live and streams telemetry over a serial/local port to a browser dashboard ("eyes in the sky").

FOCUS AREAS FOR STEP 1:

1. Review my fork's tool-call interceptor and serial dashboard stream to understand the current dataflow.

2. Review the NanoClaw routing schema (inside ECC) to map how incoming prompts are evaluated and how skills/tools are hot-loaded.

3. Compare and evaluate "mem-search" vs. "One Skill to Rule Them All" to establish the orchestrator's primary retrieval and enforcement spine.
4. Design the divergence detector to catch whenever Claude is provided with an available tool or skill but refuses or bypasses it, and trigger an immediate visual/toast red flag.

Acknowledge these constraints, and let's begin with Step 1: reviewing my fork's tool-call interceptor, the two aforementioned skills and the NanoClaw schema.